

PRZEWODNIK PRAKTYCZNY

# Lakeflow Connect

Ingestia z Azure SQL Database przez zarządzany konektor SQL Server

---

Ten przewodnik ładuje bazę Azure SQL Database do Unity Catalog za pomocą zarządzanego konektora SQL Server w Lakeflow Connect. Konektor składa się z trzech obiektów. **Połączenie Unity Catalog** (connection) przechowuje adres i poświadczenia źródła. **Brama ingestii** (ingestion gateway) to pipeline na klasycznych zasobach, który działa w sposób ciągły: pobiera początkowy snapshot, a potem strumień zmian z bazy, i zapisuje dane w wolumenie Unity Catalog. **Zarządzany pipeline ingestii** działa na serverless i nanosi zapisane dane na streaming tables w schemacie docelowym.

Źródłem na tym szkoleniu jest przykładowa baza AdventureWorksLT, tworzona przez `infra/terraform` z repozytorium szkolenia i przygotowywana skryptem `infra/sql/enable_cdc.sql`. Notebook demonstracyjny `02a_lake-flow_connect_demo` przechodzi przez te same kroki na zajęciach.

Przed rozpoczęciem potrzebujesz:

- subskrypcji Azure, w której możesz tworzyć zasoby, z rolami Contributor i User Access Administrator, a także Terraform 1.5 lub nowszego, Azure CLI i `sqlcmd` (albo Azure Data Studio)
- workspace'u Azure Databricks z włączonym Unity Catalog i zasobami serverless
- uprawnienia do tworzenia klastrów bez ograniczeń, bo brama działa na klasycznych zasobach
- w Unity Catalog: `CREATE CONNECTION` na metastore (albo `USE CONNECTION` na istniejącym połączeniu), `USE CATALOG` na katalogu docelowym oraz `USE SCHEMA`, `CREATE TABLE` i `CREATE VOLUME` na istniejących schematach lub `CREATE SCHEMA` na katalogu

Subskrypcja szkoleniowa jest współdzielona z innymi szkoleniami. Przed uruchomieniem `terraform apply` lub `terraform destroy` uzgodnij termin.

| Krok | Zadanie                                         |
|------|-------------------------------------------------|
| 1    | Utworzenie bazy źródłowej przez Terraform       |
| 2    | Włączenie CDC i utworzenie użytkownika ingestii |
| 3    | Utworzenie połączenia Unity Catalog             |
| 4    | Utworzenie bramy i pipeline'u ingestii          |
| 5    | Sprawdzenie załadowanych tabel                  |
| 6    | Test przepływu zmian (CDC)                      |
| 7    | Kontrola kosztów i sprzątanie                   |

## KROK 1

# Utworzenie bazy źródłowej przez Terraform

W `infra/terraform` skopiuj przykładowy plik zmiennych i uzupełnij go. Hasło administratora SQL ustaw przez zmienną środowiskową, a nie w pliku, i dodaj publiczny adres IP komputera, z którego uruchomisz `sqlcmd`.

```
cd infra/terraform
cp terraform.tfvars.example terraform.tfvars      # prefix, dates, trainer_ip_cidrs
export TF_VAR_sql_admin_password='<strong password>'
terraform init
terraform plan
terraform apply
```

Plik `sql_server.tf` tworzy dla Lakeflow Connect następujące zasoby:

| Zasób                                                                       | Ustawienie                                                                                                                    | Po co                                                                                                          |
|-----------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|
| <code>azurerm_mssql_server</code>                                           | TLS 1.2, administrator deatrainner                                                                                            | Serwer logiczny, na którym działa baza                                                                         |
| <code>azurerm_mssql_database</code>                                         | <code>sku_name = "GP_S_Gen5_1", min_capacity = 0.5, auto_pause_delay_in_minutes = 60, sample_name = "AdventureWorksLT"</code> | General Purpose serverless do 1 vCore: najtańszy poziom obsługujący CDC; usypia się po godzinie bez aktywności |
| <code>azurerm_mssql_firewall_rule</code><br><code>AllowAzureServices</code> | <code>0.0.0.0 do 0.0.0.0</code>                                                                                               | Pozwala bramie działającej w Azure połączyć się z serwerem                                                     |
| <code>azurerm_mssql_firewall_rule</code><br><code>trainer-NN</code>         | <code>Z trainer_ip_cidrs</code>                                                                                               | Pozwala połączyć się przez <code>sqlcmd</code> z własnego komputera                                            |

CDC w Azure SQL Database działa na każdym poziomie vCore, a w modelu DTU dopiero od S3, więc nie obniżaj SKU. Reguła *Allow Azure services* otwiera serwer dla dowolnego tenanta Azure i jest do przyjęcia tylko dla krótko żyjącej bazy szkoleniowej. Na produkcji użyj workspace'u we własnej sieci wirtualnej (VNet injection) z service endpoint lub private endpoint.

Po zakończeniu `apply` odczytaj wartości potrzebne w kolejnych krokach:

```
terraform output sql_server_fqdn          # sql-dea-training-
<suffix>.database.windows.net
terraform output sql_database_name        # AdventureWorksLT
terraform output lakeflow_connect_connection_summary # host, port 1433, database, user
```

## KROK 2

# Włączenie CDC i utworzenie użytkownika ingestii

Skrypt `infra/sql/enable_cdc.sql` włącza CDC dla bazy i dla czterech tabel w schemacie `SalesLT`: `Customer`, `Product`, `SalesOrderHeader` i `SalesOrderDetail`. Tworzy też użytkownika bazy (contained user) `lakeflow_connect` z prawem odczytu tych tabel i schematu `cdc` oraz z uprawnieniami do metadanych, których konektor potrzebuje w Azure SQL Database (`VIEW DATABASE STATE`, `SELECT` na odczytywanych widokach katalogowych `sys`, `EXECUTE` na `sp_tables`, `sp_columns_100`, `sp_pkeys` i `sp_statistics_100`). Skrypt sprawdza, co już istnieje, więc można go bezpiecznie uruchamiać wielokrotnie.

Przed uruchomieniem zastąp hasło zastępcze `CHANGE_ME-LakeflowC0nnect!` w poleceniu `CREATE USER` własnym silnym hasłem. To samo hasło wpiszesz w połączeniu w kroku 3. Następnie uruchom skrypt jako administrator SQL na bazie `AdventureWorksLT`, a nie `master`:

```
sqlcmd -S "$(terraform output -raw sql_server_fqdn)" -d AdventureWorksLT \  
-U deatrainner -P "$TF_VAR_sql_admin_password" -i ../sql/enable_cdc.sql
```

Jeśli skrypt został już uruchomiony z hasłem zastępczym, zmień je:

```
ALTER USER lakeflow_connect WITH PASSWORD = N'<your password>';
```

Sprawdź wynik. Pierwsze zapytanie powinno zwrócić 1, a procedura powinna wypisać cztery instancje przechwytywania:

```
SELECT is_cdc_enabled FROM sys.databases WHERE name = DB_NAME();  
EXEC sys.sp_cdc_help_change_data_capture;
```

## Change tracking czy CDC

Konektor odczytuje zmiany przez change tracking albo change data capture w SQL Server. Jeśli dla tabeli włączone są oba mechanizmy, używa change tracking.

|                      | Change tracking          | Change data capture                                       |
|----------------------|--------------------------|-----------------------------------------------------------|
| Rejestruje           | Fakt zmiany wiersza      | Każdą operację, z obrazem przed i po                      |
| Wymaga               | Klucza głównego          | Klucz główny niepotrzebny                                 |
| Obciążenie źródła    | Niskie                   | Wyższe: czyta dziennik transakcji i zapisuje tabele zmian |
| Poziom Azure SQL     | Dowolny                  | Poziomy vCore albo DTU od S3                              |
| Zalecenie Databricks | Tabele z kluczem głównym | Tabele bez klucza głównego                                |

Cztery tabele SalesLT mają klucze główne, więc change tracking też by zadziałał. Na szkoleniu używamy CDC, żeby było widać tabele zmian w bazie. Wariant z change tracking jest zakomentowany na końcu `enable_cdc.sql`. Dla jednej tabeli włączaj tylko jeden mechanizm.

## Dodatkowe uprawnienia

`enable_cdc.sql` nie tworzy obiektów śledzących zmiany schematu (DDL support objects). W kroku 4 przycisk **Validate** w kreatorze pokaże, czego brakuje. Najpewniej naprawia to skrypt narzędziowy Databricks `utility_script.sql`, podlinkowany na stronie dokumentacji konektora *Prepare SQL Server for ingestion using the utility objects script*. Uruchom go jako administrator w AdventureWorksLT, a następnie wykonaj:

```
EXEC dbo.lakeflowSetupChangeDataCapture @Tables = 'SCHEMAS:SalesLT', @User = 'lakeflow_connect';  
EXEC dbo.lakeflowFixPermissions @Tables = 'SCHEMAS:SalesLT', @User = 'lakeflow_connect';
```

## KROK 3

## Utworzenie połączenia Unity Catalog

W workspace Databricks otwórz **Catalog**, potem **Connect** (w niektórych workspace'ach **External data**) i **Connections**, a następnie wybierz **Create connection**. Połączenie można też utworzyć w kreatorze ingestii w kroku 4.

| Pole            | Wartość                                                                |
|-----------------|------------------------------------------------------------------------|
| Connection name | retailhub_sqlserver                                                    |
| Connection type | SQL Server                                                             |
| Host            | Wynik sql_server_fqdn, np. sql-dea-training-ab12c.database.windows.net |
| Port            | 1433                                                                   |
| User            | lakeflow_connect                                                       |
| Password        | Hasło ustawione w kroku 2                                              |

Wybierz **Create connection**. Połączenie jest teraz obiektem zabezpieczanym Unity Catalog: osoby tworzące pipeline'y ingestii potrzebują na nim `USE CONNECTION`, ale nigdy nie widzą hasła.

```
SHOW CONNECTIONS;  
DESCRIBE CONNECTION retailhub_sqlserver;  
GRANT USE CONNECTION ON CONNECTION retailhub_sqlserver TO `data_engineers`;
```

## KROK 4

# Utworzenie bramy i pipeline'u ingestii

Na pasku bocznym workspace'u otwórz **Data Ingestion** i w sekcji **Databricks connectors** wybierz **SQL Server**. Kreator utworzy oba pipeline'y.

| Strona kreatora             | Ustawienie                | Wartość na szkoleniu                                                                 |
|-----------------------------|---------------------------|--------------------------------------------------------------------------------------|
| Connection                  | Połączenie                | retailhub_sqlserver                                                                  |
| Ingestion setup             | Nazwa pipeline'u ingestii | retailhub_sqlserver_ingestion                                                        |
|                             | Lokalizacja event logu    | retailhub_trainer.lakeflow_staging                                                   |
|                             | Nazwa bramy ingestii      | retailhub_sqlserver_gateway                                                          |
|                             | Staging location          | Katalog retailhub_trainer, schemat lakeflow_staging; tu brama tworzy swój wolumen    |
| Source                      | Tabele                    | AdventureWorksLT › SalesLT › Customer, Product, SalesOrderHeader, SalesOrderDetail   |
|                             | History tracking          | Wyłączone (SCD typu 1): wiersze docelowe są aktualizowane i usuwane tak jak w źródle |
| Destination                 | Katalog i schemat         | retailhub_trainer.lakeflow_connect                                                   |
| Database setup              | Validate                  | Przed przejściem dalej popraw wszystko, co zostanie zgłoszone (zob. krok 2)          |
| Schedules and notifications | Harmonogram               | Brak na potrzeby demo; na produkcji np. co 30 minut                                  |
|                             | Powiadomienia             | Twój e-mail przy błędzie                                                             |

Wybierz **Save and run pipeline**. W **Jobs & Pipelines** zobaczysz dwa pipeline'y:

- **Brama** uruchamia klasyczne zasoby, zapisuje w wolumenie roboczym początkowy snapshot czterech tabel, a potem stale czyta strumień zmian. Jest przeznaczona do pracy ciągłej.
- **Pipeline ingestii** działa w trybie wyzwalanych aktualizacji na serverless. Każda aktualizacja nanosi zapisany snapshot i zmiany na docelowe streaming tables.

Pierwszy snapshot trwa kilka minut. Widok listy pipeline'u ingestii pokazuje, ile rekordów przetworzono dla każdej tabeli; kolumny *Upserted records* i *Deleted records* można dodać w ustawieniach kolumn. Jeden pipeline obsługuje do 250 tabel, a każdy pipeline ingestii ma dokładnie jedną bramę.

Nadaj uczestnikom szkolenia prawo odczytu miejsca docelowego:

```
GRANT USE CATALOG ON CATALOG retailhub_trainer TO `dea-training`;  
GRANT USE SCHEMA, SELECT ON SCHEMA retailhub_trainer.lakeflow_connect TO `dea-training`;
```

## KROK 5

## Sprawdzenie załadowanych tabel

---

Po zakończeniu pierwszej aktualizacji pipeline'u ingestii uruchom poniższe zapytania w notebooku lub edytorze SQL. Unity Catalog przechowuje nazwy tabel małymi literami, więc SalesOrderHeader staje się salesorderheader.

```
SHOW TABLES IN retailhub_trainer.lakeflow_connect;

SELECT 'customer' AS table_name, COUNT(*) AS row_count FROM
retailhub_trainer.lakeflow_connect.customer
UNION ALL SELECT 'product', COUNT(*) FROM retailhub_trainer.lakeflow_connect.product
UNION ALL SELECT 'salesorderheader', COUNT(*) FROM
retailhub_trainer.lakeflow_connect.salesorderheader
UNION ALL SELECT 'salesorderdetail', COUNT(*) FROM
retailhub_trainer.lakeflow_connect.salesorderdetail;
```

```
-- Top products by wholesale revenue
SELECT p.Name AS product_name, SUM(d.OrderQty) AS units, ROUND(SUM(d.LineTotal), 2) AS revenue
FROM retailhub_trainer.lakeflow_connect.salesorderdetail d
JOIN retailhub_trainer.lakeflow_connect.product p ON d.ProductID = p.ProductID
GROUP BY p.Name
ORDER BY revenue DESC
LIMIT 10;
```

```
DESCRIBE TABLE EXTENDED retailhub_trainer.lakeflow_connect.customer; -- streaming table owned by
the pipeline
DESCRIBE HISTORY retailhub_trainer.lakeflow_connect.customer LIMIT 5;
```

Porównaj liczby wierszy z wynikiem `SELECT COUNT(*)` na tabelach źródłowych w Azure SQL. Typy danych są konwertowane automatycznie: na przykład `MONEY` staje się `DECIMAL(19,4)`, `DATETIME` staje się `TIMESTAMP`, a `UNIQUEIDENTIFIER` i `XML` stają się `STRING`.

## KROK 6

## Test przepływu zmian (CDC)

---

W Databricks zapisz bieżącą wartość:

```
SELECT CustomerID, FirstName, LastName, Phone, ModifiedDate
FROM retailhub_trainer.lakeflow_connect.customer
WHERE CustomerID = 1;
```

Zmień wiersz w Azure SQL, w Query editor w Azure Portal lub przez `sqlcmd`, i sprawdź, czy CDC przechwyciło zmianę. W Azure SQL Database zadanie przechwytywania działa mniej więcej co 20 sekund.

```

UPDATE SalesLT.Customer
SET    Phone = '555-TRAINING-42', ModifiedDate = SYSDATETIME()
WHERE  CustomerID = 1;

SELECT __$operation, CustomerID, Phone
FROM   cdc.SalesLT_Customer_CT
WHERE  CustomerID = 1;          -- operation 3 = before update, 4 = after update

```

W **Jobs & Pipelines** otwórz pipeline ingestii i wybierz **Start**. Brama ma już zapisaną zmianę, a aktualizacja scala ją z tabelą docelową. Uruchom ponownie zapytanie w Databricks: **Phone** ma teraz wartość **555-TRAINING-42**. Wstawienia i usunięcia w źródle trafiają do miejsca docelowego w ten sam sposób; przy wyłączonym history tracking wiersz usunięty w źródle znika z tabeli docelowej.

## KROK 7

# Kontrola kosztów i sprzątanie

| Składnik                 | Za co płacisz                                 | Jak ograniczyć                                                                                                                   |
|--------------------------|-----------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| Brama ingestii           | Klasyczne VM i DBU przez cały czas działania  | Największy koszt w tym układzie; usuń bramę zaraz po demo                                                                        |
| Pipeline ingestii        | DBU serverless za każdą aktualizację          | Uruchamiaj ręcznie lub rzadko według harmonogramu                                                                                |
| Azure SQL Database       | Sekundy vCore w czasie działania plus magazyn | Auto-pause po 60 minutach bezczynności; działająca brama utrzymuje bazę aktywną, a uśpiona baza nie przechwytyje zmian przez CDC |
| Wolumen roboczy i tabele | Magazyn                                       | Usuń schematy po zakończeniu                                                                                                     |

Dokumentacja Databricks ostrzega przed zatrzymywaniem bramy na produkcji. Jeśli brama stoi dłużej niż okres przechowywania zmian w źródle (dla CDC domyślnie trzy dni), zmiany przepadają i tabele wymagają pełnego odświeżenia. Na krótkim szkoleniu nie ma to znaczenia, więc usuń wszystko zaraz po demo:

1. W **Jobs & Pipelines** usuń pipeline ingestii, a potem bramę.
2. Usuń obiekty Databricks utworzone przez pipeline'y i przez Ciebie:

```

DROP SCHEMA IF EXISTS retailhub_trainer.lakeflow_connect CASCADE;
DROP SCHEMA IF EXISTS retailhub_trainer.lakeflow_staging CASCADE;
DROP CONNECTION IF EXISTS retailhub_sqlserver;

```

3. Po ostatnim dniu szkolenia usuń zasoby Azure i sprawdź, że grupa zasobów zniknęła:

```

cd infra/terraform
terraform destroy
az group list --o table | grep dea

```



## Podsumowanie

| Gdzie                        | Krok                                               | Wynik                                                                                    |
|------------------------------|----------------------------------------------------|------------------------------------------------------------------------------------------|
| Terminal, Terraform          | 1. <code>terraform apply</code>                    | Serwer Azure SQL, baza serverless AdventureWorksLT, reguły firewalla, wartości wyjściowe |
| sqlcmd                       | 2. <code>enable_cdc.sql</code> z własnym hasłem    | CDC na czterech tabelach SalesLT, użytkownik lakeflow_connect                            |
| Catalog, Connections         | 3. Create connection                               | Połączenie Unity Catalog retailhub_sqlserver                                             |
| Data Ingestion, SQL Server   | 4. Kreator                                         | Brama, wolumen roboczy, pipeline ingestii, uprawnienia dla uczestników                   |
| Edytor SQL lub notebook      | 5. Zapytania kontrolne                             | Cztery streaming tables w retailhub_trainer.lakeflow_connect                             |
| Azure SQL i Jobs & Pipelines | 6. UPDATE, potem Start                             | Zmiana widoczna w tabeli docelowej                                                       |
| Jobs & Pipelines, Terraform  | 7. Usunięcie, DROP, <code>terraform destroy</code> | Brak pozostawionych zasobów                                                              |

## Rozwiązywanie problemów

| Objaw                                                                     | Przyczyna i rozwiązanie                                                                                                                                                                                                     |
|---------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Przekroczony czas logowania sqlcmd                                        | Twojego adresu IP nie ma w <code>trainer_ip_cidrs</code> (sprawdź przez <code>curl -s ifconfig.me</code> , dodaj i ponownie wykonaj <code>apply</code> ) albo baza serverless właśnie się wybudza; spróbuj po około minucie |
| Test połączenia lub brama nie dociera do serwera                          | Brak reguły firewalla <code>AllowAzureServices</code> albo błędny host lub port; host musi być pełną nazwą <code>*.database.windows.net</code> , a port to 1433                                                             |
| Pierwsze połączenie po przerwie trwa długo lub kończy się timeoutem       | Baza się uśpiła; wybudzenie trwa 30-60 sekund; wykonaj zapytanie tuż przed demo                                                                                                                                             |
| <code>sp_cdc_enable_db</code> kończy się błędem                           | Połączenie z master (użyj <code>-d AdventureWorksLT</code> ) albo poziom poniżej 1 vCore lub S3; zostaw <code>GP_S_Gen5_1</code> albo przejdź na change tracking                                                            |
| Walidacja zgłasza brak CDC lub change tracking albo brak uprawnień        | Uruchom ponownie <code>enable_cdc.sql</code> ; sprawdź <code>SELECT is_tracked_by_cdc FROM sys.tables</code> ; uruchom skrypt narzędziowy Databricks z <code>lakeflowFixPermissions</code>                                  |
| <code>PERMISSION_DENIED: You are not authorized to create clusters</code> | Użytkownik nie ma uprawnień do tworzenia klastrów bez ograniczeń, którego wymaga brama; musi je nadać administrator workspace'u                                                                                             |
| Kreator nie pozwala utworzyć połączenia, wolumenu lub tabel               | Brak uprawnień Unity Catalog: <code>CREATE CONNECTION</code> , <code>USE CATALOG</code> , <code>CREATE SCHEMA</code> lub <code>USE SCHEMA</code> , <code>CREATE TABLE</code> , <code>CREATE VOLUME</code>                   |
| Zapytanie zgłasza, że tabela nie istnieje                                 | Użyj nazw małymi literami ( <code>salesorderheader</code> ) i schematu docelowego, a nie roboczego; poczekaj na koniec pierwszej aktualizacji                                                                               |
| <code>Cannot have multiple queries named ...</code>                       | Dwa schematy źródłowe mają tabele o tej samej nazwie kierowane do jednego schematu docelowego; ustaw różne nazwy docelowe lub użyj osobnych par bramy i pipeline'u                                                          |
| Zmiana ze źródła się nie pojawia                                          | Zadanie przechwytywania jeszcze nie ruszyło (około 20 sekund), po zmianie nie było aktualizacji ingestii albo baza była uśpiona                                                                                             |
| <code>NON_INCREMENTAL_SCHEMA_CHANGE</code> po dodaniu kolumny w źródle    | Wykonaj pełne odświeżenie tej tabeli ze strony pipeline'u                                                                                                                                                                   |